Persistance Layer

Hibernate, JPA, JDBC & Database – Clean Notes

1. JPA (Java Persistence API)

What is JPA

JPA is a specification (rules) that defines how Java objects should be stored and retrieved from a database.
It does not contain actual code to talk to the database.

Type

- Interface + Annotations
- Not an implementation

Why JPA is Needed

- Removes direct JDBC code
- Provides standard ORM rules
- Makes code database independent

How We Get JPA

- Added via **spring-boot-starter-data-jpa** dependency

Important Interfaces

- EntityManager
- EntityTransaction
- JPARepository
  JPA only defines **what should happen**, not **how**.

Core JPA Interfaces

The most important JPA interfaces are:

- **EntityManager** – performs database operations
- **EntityTransaction** – manages transactions
- **Query / TypedQuery** – executes JPQL queries

These interfaces are **implemented by Hibernate**.

CRUD Operations in JPA

JPA supports CRUD operations without SQL:

- Create → persist()
- Read → find()
- Update → merge()
- Delete → remove()

In Spring Boot, these are commonly accessed through repositories.

Repository Interface (Spring Data JPA)

A repository interface is used to perform database operations on an entity.
Example:

```
public interface EmployeeRepository
    extends JpaRepository<EmployeeEntity, Long> {
```

}
Spring automatically generates the implementation at runtime.

CRUD via JpaRepository

JpaRepository provides ready-made methods:

- save() – insert or update
- findById() – fetch by ID
- findAll() – fetch all records
- deleteById() – delete record

No implementation code is required.

Important JPA Annotations

- @Entity – marks a class as a database table
- @Table – specifies table name
- @Id – defines primary key
- @GeneratedValue – auto-generates ID

Every entity must have exactly one @Id.

---

2. Hibernate

What is Hibernate

Hibernate is an **ORM framework** and the **implementation of JPA**.
It converts Java objects into SQL queries and executes them.

Type

- Concrete classes
- Implements JPA interfaces

Why Hibernate is Needed

- Generates SQL automatically
- Manages entity lifecycle
- Handles caching and transactions

How We Get Hibernate

- Automatically included with spring-boot-starter-data-jpa

What Hibernate Does Internally

- Reads @Entity, @Table, @Column
- Generates SQL
- Uses JDBC to execute SQL

Example

```
@Entity
@Table(name = "employee")
public class EmployeeEntity {
   @Id
   @GeneratedValue
   private Long id;
}
```

You never write SQL, Hibernate does it.

---

3. JDBC (Java Database Connectivity)

What is JDBC

JDBC is a **low-level Java API** used to execute SQL queries.

Type

- Interfaces and classes
- Part of Java SE

Why JDBC is Needed

- Database understands only SQL
- Hibernate uses JDBC internally

Who Uses JDBC

- Hibernate (not you)
- Spring internally

Example (Pure JDBC)

```
Connection con = DriverManager.getConnection(url, user, pass);
PreparedStatement ps = con.prepareStatement("INSERT INTO employee VALUES(?)");
ps.executeUpdate();
```

Hibernate hides this code.

---

4. JDBC Driver

What is JDBC Driver

A JDBC Driver is a **vendor-specific implementation** that converts Java calls into database commands.

Examples

- MySQL Driver
- PostgreSQL Driver

How We Get Driver

Added automatically or manually in pom.xml

<dependency>

<groupId>mysql</groupId>

<artifactId>mysql-connector-j</artifactId>

</dependency>

Without driver:

- No database connection possible

---

5. Database

What is Database

A database stores data in **tables, rows, and columns**.

Role

- Executes SQL
- Stores persistent data
- Returns result to JDBC driver

Hibernate never talks to DB directly.

---

6. Flow of Execution (Short & Clear)

Repository

↓

JPA (rules)

↓

Hibernate (ORM implementation)

↓

JDBC (SQL execution)

↓

JDBC Driver

↓

Database

And response comes back the same path.

---

7. Who Does What (One Line Each)

- **JPA**: Defines rules
- **Hibernate**: Generates and manages SQL
- **JDBC**: Executes SQL
- **Driver**: Talks to DB
- **Database**: Stores data

---

8. Simple Save Example (Spring Boot)

employeeRepository.save(employee);

Behind the scenes:

- JPA decides action
- Hibernate generates SQL
- JDBC executes
- Driver sends to DB

You Are Now Clear Up To:

✓ ORM
✓ JPA vs Hibernate
✓ JDBC role
✓ Execution flow

spring-boot-starter-data-jpa – In Short Notes

spring-boot-starter-data-jpa provides **JPA and Hibernate integration** in Spring Boot.
It includes Hibernate ORM, JPA interfaces, transaction management, and repository support.
It allows developers to perform CRUD operations without writing SQL or JDBC code.
Repositories like JpaRepository are auto-implemented at runtime by Spring.
This dependency simplifies database interaction using annotations and repositories.

H2 Database

H2 is a **lightweight, in-memory database** mainly used for **development and testing**.
It starts automatically with the application and does not require separate installation.
Data is stored in memory and is lost when the application stops (unless file mode is used).
H2 provides a **web console** to view tables and data in the browser.
It is fast, easy to configure, and ideal for learning JPA and Spring Boot.

Application.Properties

- Steps To Envoke H2 Console

1. In newer Spring Boot (4.x), the H2 Console is **not auto-enabled** by default for security and clarity reasons.
2. Since the H2 Console runs as a **web servlet**, Spring Boot now requires **explicit servlet registration** instead of hidden defaults.
3. A separate configuration class (H2ConsoleConfig) is created and annotated with @Configuration to enable custom setup.
4. Inside this class, a ServletRegistrationBean<JakartaWebServlet> bean is defined to manually register the H2 console servlet.
5. The servlet used is JakartaWebServlet, which is provided by the H2 database to render the web-based console UI.
6. The servlet is mapped to the URL path /h2-console/* so the console can be accessed via browser.
7. setLoadOnStartup(1) ensures the H2 console servlet is loaded when the application starts.
8. ServletRegistrationBean allows servlet registration **without using web.xml**, following modern Spring Boot practices.
9. This configuration makes the H2 console available **only when explicitly enabled**, giving developers full control over exposure.
10. The setup requires adding both com.h2database:h2 and spring-boot-starter-web dependencies to the project.

1. Use in a Real-World Project

In a **Patient Management System**:

- **@Id:** Ensures every patient has a unique Patient ID (e.g., #501). Even if two patients are named "Rahul", the system distinguishes them by this ID.
- **@GeneratedValue:** When a receptionist registers a new patient, they don't manually assign "Patient #502". The system automatically calculates and assigns the next available number.

2. Short Notes (The Concepts)

A. @Id (The Identifier)

- **Purpose:** Marks a field as the **Primary Key** of the database table.
- **Rule:** Every Entity class **must** have one @Id.
- **Function:** It guarantees that every **row** (record) in the table is unique.

B. @GeneratedValue (The Automation)

- **Purpose:** Tells the database to **automatically generate** values for the ID column.
- **Default:** If omitted, you must manually set the ID (e.g., user.setId(55)).
- **Key Attribute:** strategy (Defines *how* the number is created).

3. Parameters (Generation Strategies)

These are the values you pass to strategy = GenerationType.X.

1. **GenerationType.IDENTITY**
    - **Logic:** Uses the database's internal **Auto-Increment** feature.
    - **Best For: MySQL**, MariaDB, SQL Server.
    - **Note:** The ID is generated *after* the data is inserted.
2. **GenerationType.SEQUENCE**
    - **Logic:** Uses a separate database object (Sequence) to count numbers.
    - **Best For: PostgreSQL**, Oracle.
    - **Note:** High performance; allows fetching IDs *before* inserting data (good for batching).
3. **GenerationType.AUTO**
    - **Logic:** Lets the framework (Hibernate) pick the best strategy for your specific database.
    - **Best For:** Prototyping or when the database type is unknown.

4. Small Program

Java

```
import jakarta.persistence.*;


@Entity
public class Student {

  // 1. Marks this as the Unique Key (Roll Number)
  @Id

  // 2. Automates the numbering using MySQL's Auto-Increment
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  private String name;
}
```

5. Short Story: "The Classroom Roll Call"

Imagine a classroom.

- @Id (The Roll Number):

There are two students named "Rahul".

The teacher gives every student a Roll Number.

- Rahul A is **Roll No. 1**.
- Rahul B is Roll No. 2.

Now, when the teacher calls "Roll No 2", only one person stands up. That unique number is the @Id.

- @GeneratedValue (The Admission Counter):

A new student joins. The teacher doesn't have to remember that the last student was #50.

The school's computer automatically assigns #51 to the new student. The teacher just types the name, and the ID appears automatically.

Next Step:

Now that your data is ready (Entity), would you like to build the tool that saves it? Let's write the Repository Interface.